07

Part 7 · Architecture & DevOps

Application styles and the automation that builds, ships and observes them.

- Monolith vs Microservices
- APIs
- Queues & events
- Monitoring vs Logging
- Infrastructure as Code
- CI/CD
- Edge Computing
- CDN

7.1 Microservices vs Monolithic Architecture

★★★

IN ONE LINE

One deployable application versus many small, independently deployable services.

A **monolith** packages the whole application — UI, business logic, data access — as **one deployable unit**. **Microservices** split it into **small, independent services**, each owning one business capability and its own data, communicating over APIs or events.

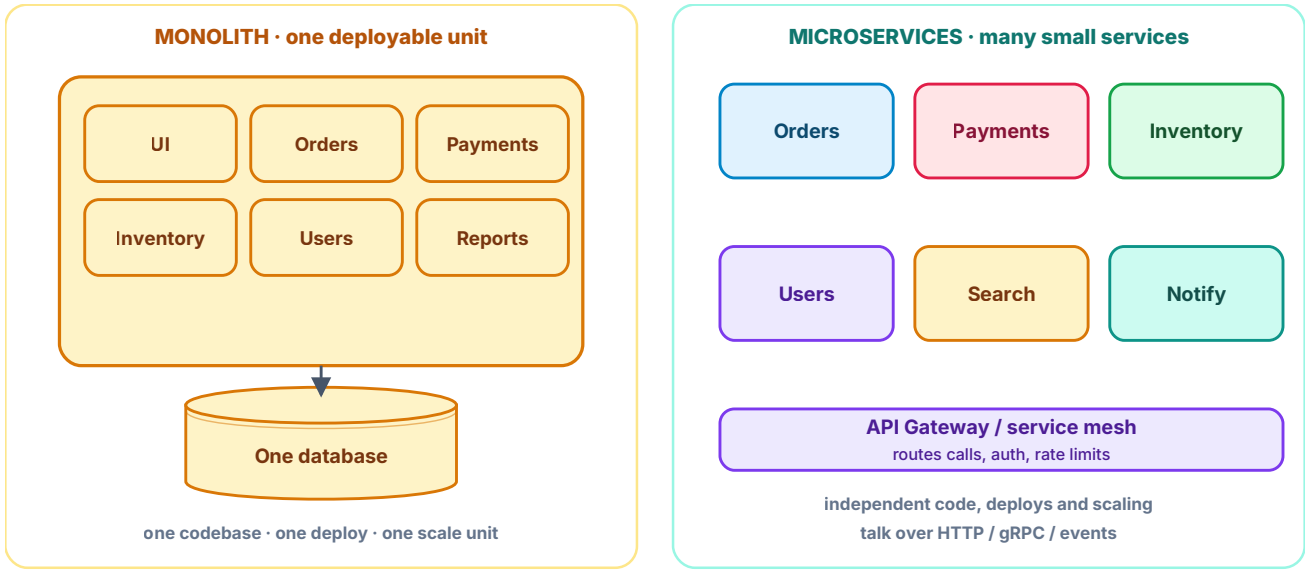


Figure 7.1 — One big unit vs many small services

	Monolith	Microservices
Deployment	All-or-nothing release	Each service deployed independently
Scaling	Scale the entire app	Scale only the busy service
Technology	One stack	Different language / DB per service
Fault isolation	A bug can bring down everything	A failure is contained (with good design)
Team structure	One large team, tight coupling	Small autonomous teams
Complexity	**Simple** to build and debug at first	Distributed-systems complexity: networking, tracing, data consistency
Best for	Small teams, early products, simple domains	Large systems, many teams, uneven scaling needs

INTERVIEW ANGLE

Do not claim microservices are always better. A grown-up answer: **"start with a well-structured monolith, split into services when team size and scaling needs justify the extra operational cost."** Cloud makes microservices easier via containers, Kubernetes, API gateways, serverless and managed queues.

REAL-WORLD ANALOGY

A single food-court stall that cooks everything (monolith) vs a food court with specialised counters: pizza, noodles, dessert (microservices). Busy pizza counter? Add staff only there.

7.2 API (Application Programming Interface)

★★★

IN ONE LINE

A defined interface through which software systems request services from each other, usually REST + JSON over HTTP.

An **API (Application Programming Interface)** is a contract that lets one piece of software ask another to do something without knowing how it works inside. In the cloud, **everything is an API**: creating a VM, uploading to a bucket, calling a model. Most web APIs follow **REST** over HTTP and exchange **JSON**.

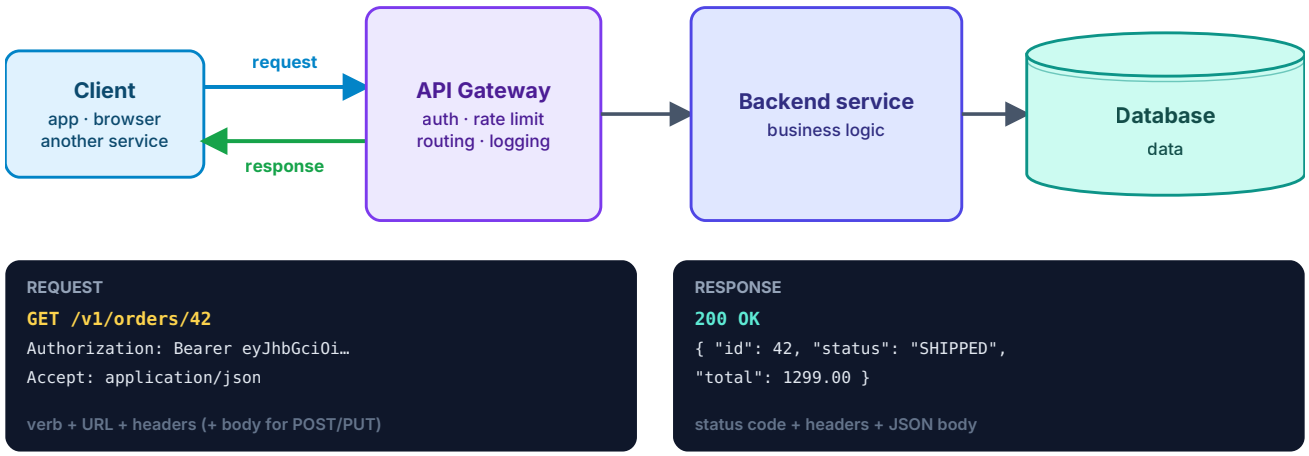


Figure 7.2 — Client, API gateway, backend, and a REST request / response

REST verb	Action	Idempotent?	Example
GET	Read	Yes	`GET /orders/42`
POST	Create	No	`POST /orders`
PUT	Replace	Yes	`PUT /orders/42`
PATCH	Partial update	Usually	`PATCH /orders/42`
DELETE	Remove	Yes	`DELETE /orders/42`

API gateway features

- Authentication and authorisation
- **Rate limiting** / throttling
- Request routing, caching, versioning
- Logging and metrics

REST vs GraphQL vs gRPC

- **REST**: resources + HTTP verbs, universal.
- **GraphQL**: client chooses exact fields, one endpoint.
- **gRPC**: binary, fast, service-to-service.

PYTHON · CALLING A REST API

```python
import requests

r = requests.get('https://api.example.com/v1/orders/42',
                 headers={'Authorization': 'Bearer <token>'}, timeout=10)
r.raise_for_status()          # 4xx / 5xx -> exception
order = r.json()
print(order['status'])
```

DATA ENGINEERING TIE-IN

Data engineers constantly **ingest from APIs**: handle pagination, rate limits (back off and retry), authentication, incremental pulls and idempotent writes.

7.3 Event-Driven Architecture & Message Queues

★★☆

IN ONE LINE

Asynchronous messaging that decouples producers from consumers and absorbs spikes.

Instead of services calling each other directly and waiting (**synchronous**), they can exchange **messages or events** through a broker (**asynchronous**). This **decouples** the sender from the receiver, smooths traffic spikes and makes systems more resilient. It is the glue of microservices and serverless architectures.

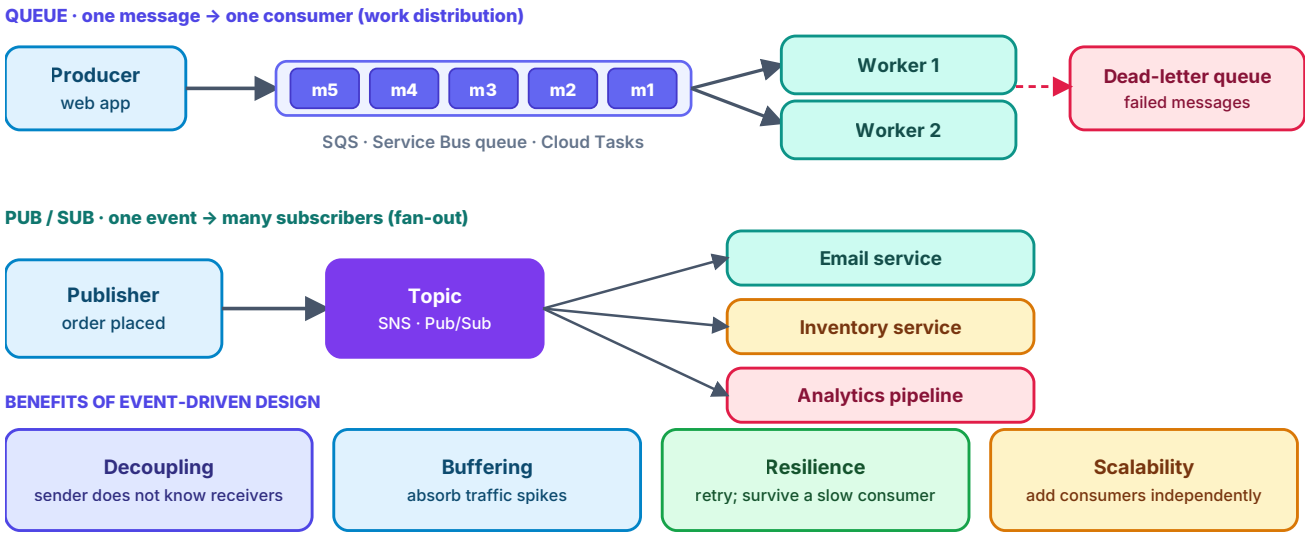


Figure 7.3 — Queue (work distribution) vs publish/subscribe (fan-out)

	Message queue	Pub / Sub topic	Event stream / log
Delivery	Each message to **one** consumer	Each message to **all** subscribers	Consumers read at their own offset; replayable
Retention	Until processed	Short, until delivered	Hours to forever
Use for	Task / job queues	Notifications, fan-out events	Analytics, CDC, event sourcing
Examples	SQS, Azure Service Bus, RabbitMQ	SNS, Event Grid, Google Pub/Sub	Kafka, Kinesis, Event Hubs

INTERVIEW ANGLE

Know these delivery words: **at-least-once** (duplicates possible, so consumers must be **idempotent**), **dead-letter queue** for poison messages, and **back-pressure**. This connects directly to exactly-once and idempotency in the Data Engineering handbook.

7.4 Monitoring vs Logging

★★★

IN ONE LINE

Monitoring watches metrics to detect problems; logging records events to explain them.

Monitoring continuously collects and watches **metrics** to tell you **whether the system is healthy and when it is not** (alerts and dashboards). **Logging** records **detailed events** so you can find out **what happened and why**. Together with **tracing** they form **observability**.

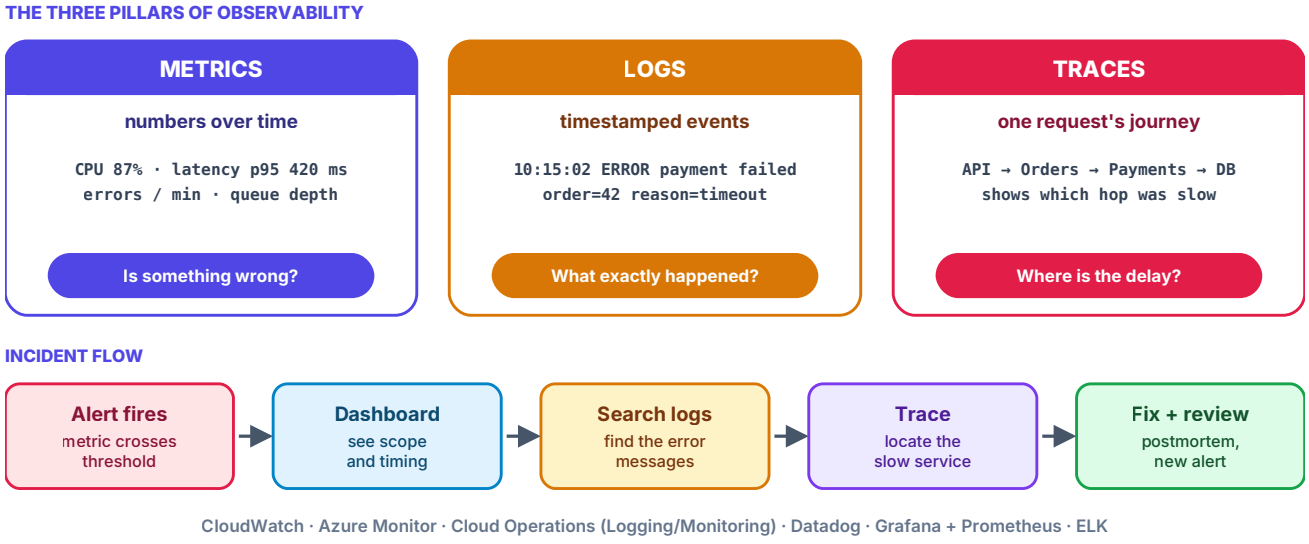


Figure 7.4 — Metrics, logs, traces and how an incident is investigated

	Monitoring	Logging
Data	Numeric metrics sampled over time	Text / structured event records
Answers	"Is it healthy right now? Is it getting worse?"	"What happened at 10:15 for order 42?"
Volume	Small, aggregated	Large, detailed (cost!)
Used for	Dashboards, alerts, auto scaling, SLOs	Debugging, auditing, security forensics
Cloud tools	CloudWatch metrics, Azure Monitor, Cloud Monitoring	CloudWatch Logs, Log Analytics, Cloud Logging, ELK

Good alerting

- Alert on **symptoms users feel** (errors, latency), not every CPU blip.
- Every alert must be **actionable** with an owner.
- Define severity and escalation paths.
- Avoid alert fatigue.

Golden signals

- **Latency** — how long requests take
- **Traffic** — demand on the system
- **Errors** — rate of failures
- **Saturation** — how full the resources are

DATA ENGINEERING TIE-IN

For pipelines monitor **freshness, row counts, failures, runtime and cost**; log each run with run-id and input window so you can debug and backfill. See data observability in the Data Engineering handbook.

7.5 Infrastructure as Code (IaC)

★★★

IN ONE LINE

Managing infrastructure through version-controlled definition files instead of manual clicks.

Infrastructure as Code (IaC) means defining servers, networks, databases and permissions in **text files** that a tool turns into real cloud resources. Infrastructure becomes **versioned, reviewable, repeatable** and testable like application code.

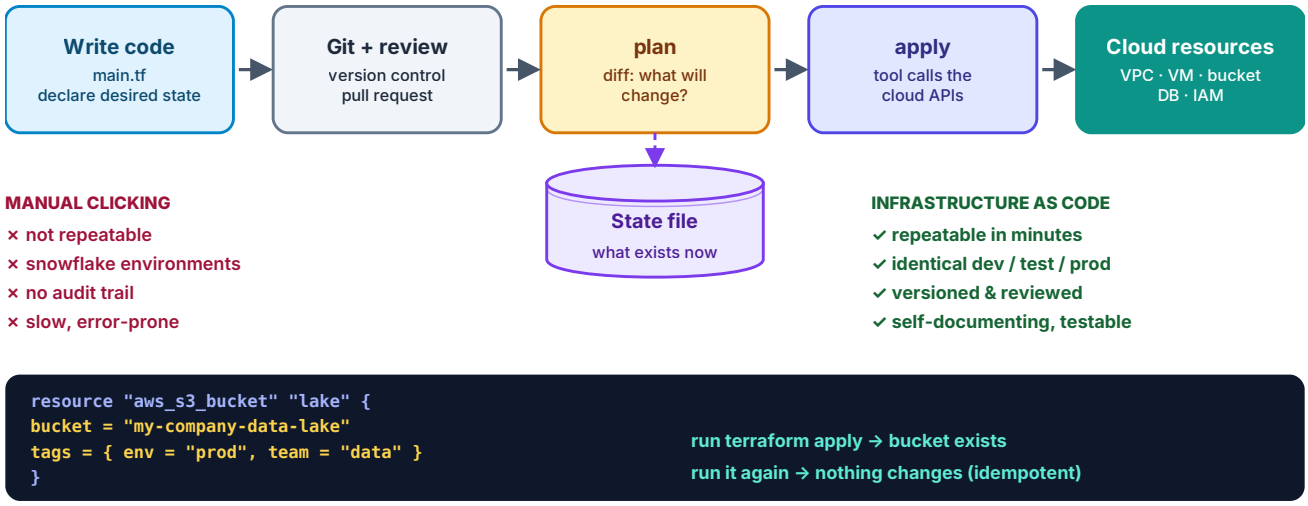


Figure 7.5 — From code to cloud: write, review, plan, apply

Concept	Meaning
Declarative	You describe the *end state* ("3 servers"); the tool works out the steps. Terraform, CloudFormation, Bicep.
Imperative	You script the *steps* ("create, then attach"). Shell scripts, SDK code, some Pulumi usage.
State	The tool's record of what it created; stored remotely and locked for teams.
Drift	Reality differs from code (someone clicked in the console). `plan` shows it.
Modules	Reusable building blocks (e.g. a standard VPC).
Idempotency	Applying the same code twice changes nothing the second time.

Tools

- **Terraform / OpenTofu** — multi-cloud, widely used
- **AWS CloudFormation / CDK**
- **Azure Bicep / ARM**
- **Pulumi** — real programming languages
- **Ansible** — configuration management of what runs *on* servers

Why companies insist on it

- Rebuild an environment (or DR region) in minutes.
- Consistency between dev, test, prod.
- Code review and audit trail for infra changes.
- Enables CI/CD for infrastructure.

INTERVIEW ANGLE

Common question: **Terraform vs CloudFormation**. Terraform is provider-agnostic (helps with multi-cloud and lock-in, 8.2); CloudFormation is AWS-native with deep integration and no separate state to manage.

7.6 CI/CD

★★★

IN ONE LINE

Automating building, testing and releasing software: continuous integration, delivery and deployment.

CI/CD automates the path from code change to running software. **Continuous Integration (CI)**: developers merge small changes frequently and every change is automatically built and tested. **Continuous Delivery (CD)**: every passing change is automatically prepared for release, with a **manual approval** to go live. **Continuous Deployment (CD)**: it goes live **automatically**.

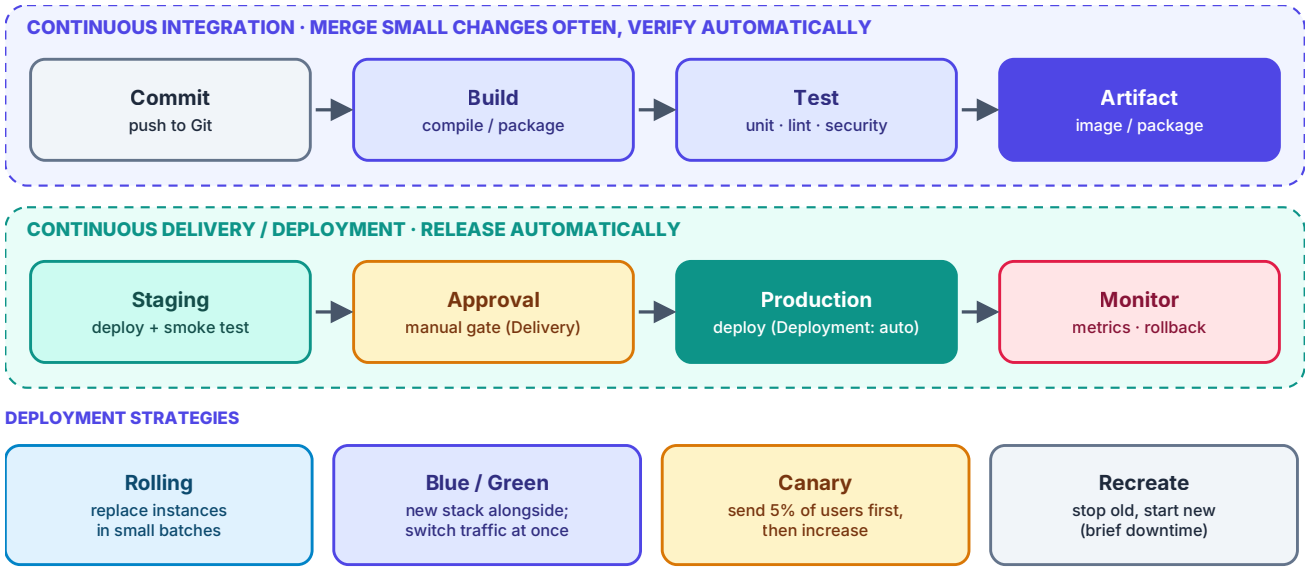


Figure 7.6 — A CI/CD pipeline and deployment strategies

	CI	Continuous Delivery	Continuous Deployment
Automates	Build + test on every commit	Release-ready artifact, deploy on button press	Deploy to production with no human step
Human gate	Code review	Yes, before production	No (relies on strong tests + monitoring)
Goal	Find bugs early	Always releasable	Fast, frequent releases

Tools

- **GitHub Actions**, GitLab CI, Jenkins
- **AWS CodePipeline / CodeBuild**, Azure DevOps / Pipelines, Google Cloud Build
- Argo CD / Flux for GitOps on Kubernetes

Good practices

- Small, frequent changes.
- Automated tests at every stage.
- Same artifact promoted through environments.
- Automatic **rollback** on failure.
- Secrets from a vault, not the pipeline file.

DATA ENGINEERING TIE-IN

Data pipelines need CI/CD too: lint SQL, unit-test transformations, run dbt tests in an isolated schema, deploy jobs via IaC / asset bundles, and promote dev → staging → prod. Details in the Data Engineering handbook (3.13).

7.7 Edge Computing

★★☆

IN ONE LINE

Processing data close to its source to cut latency and bandwidth.

Edge computing moves computation and data storage **closer to where data is generated or used** — devices, stores, factories, cell towers — instead of sending everything to a distant central cloud. It cuts **latency**, saves **bandwidth**, allows **offline operation** and can keep sensitive data local.

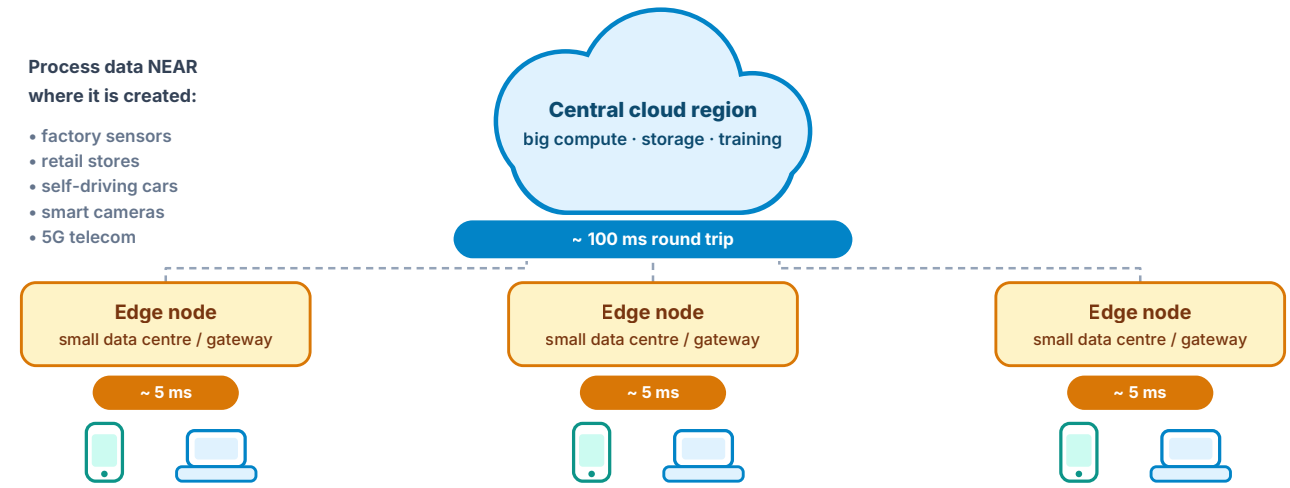


Figure 7.7 — Processing near the source vs in a central region

	Centralised cloud	Edge computing
Location	Large regional data centres	Near devices / users
Latency	Tens to hundreds of ms	Few ms
Bandwidth	Ships all raw data	Filters / aggregates locally, sends summaries
Connectivity	Needs a reliable link	Works with intermittent links
Best for	Heavy analytics, training, storage	Real-time decisions, IoT, video analytics

Examples of edge services

- AWS Outposts, Wavelength, IoT Greengrass
- Azure Stack Edge, IoT Edge
- Google Distributed Cloud
- CDN edge functions (CloudFront Functions, Cloudflare Workers)

Edge + cloud together

Typical pattern: devices → edge gateway (filter, react in ms) → cloud (store, train models, dashboards) → updated model pushed back to the edge.

REMEMBER

Edge does not replace the cloud; it **extends** it. Related: **fog computing** (an intermediate layer), and **CDN** (the most common form of edge delivery, next topic).

7.8 CDN (Content Delivery Network)

★★★

IN ONE LINE

A global network of edge caches that delivers content from the location nearest the user.

A **CDN (Content Delivery Network)** is a globally distributed network of **edge servers that cache copies of your content** close to users. Requests are served from the nearest edge instead of travelling to the origin server, giving faster loads, lower origin load and protection from traffic spikes and attacks.

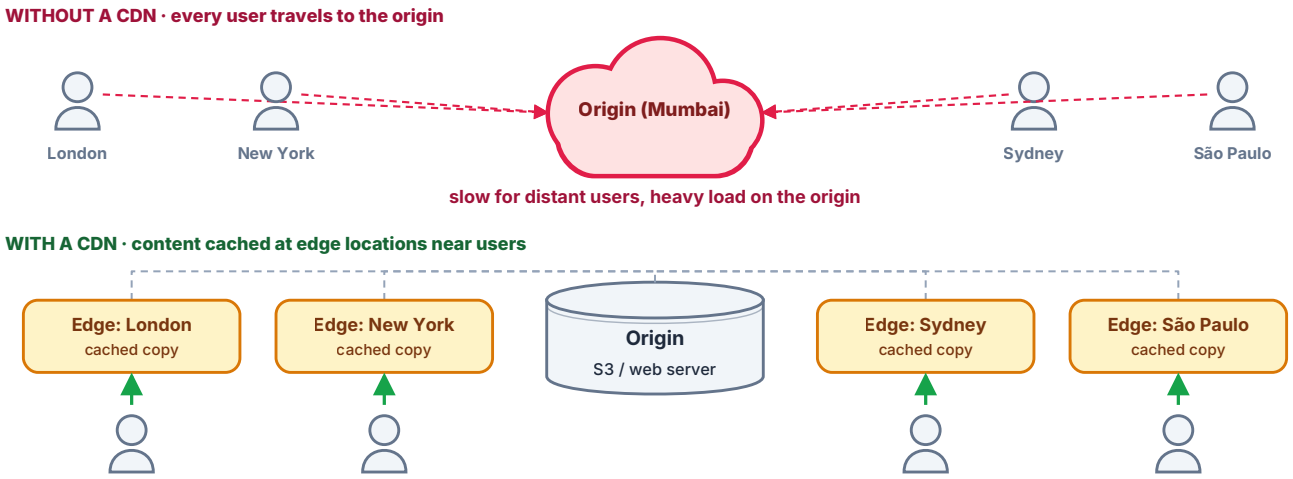


Figure 7.8a — Origin-only delivery vs delivery through edge caches

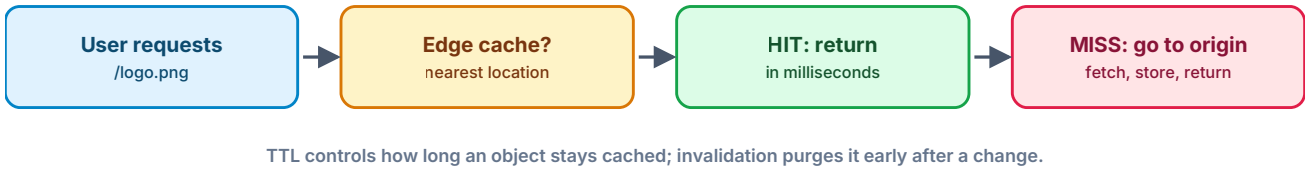


Figure 7.8b — Cache hit and cache miss

Benefit	How
Lower latency	Content served from a nearby edge location.
Reduced origin load & cost	Most requests never reach the origin; less egress from the origin.
Scalability	Edge network absorbs flash crowds (sales, live events).
Security	DDoS absorption, WAF at the edge, TLS termination.
Availability	Can serve cached content even if the origin is briefly down.

What to cache

Static content: images, CSS/JS, video segments, downloads. Dynamic or personalised pages can be cached briefly or accelerated by optimised routes.

Typical setup

Static site in an **object-storage bucket** (origin) → **CDN** in front → **HTTPS certificate** + custom domain via **DNS**. Ties together 3.1, 4.2, 4.3.

INTERVIEW ANGLE

CDN vs edge computing: a CDN primarily **caches and delivers** content near users; edge computing **runs code** near users or devices. Many CDNs now offer both (edge functions).